Exempel – klasser för textsträngar

Mutable respektive immutable

Varför dessa exempel?

- När du skriver egen kod kommer du naturligtvis att använda Javas egna klasser
  - java.lang.String
  - java.util.StringBuilder
  - Exempel på hur dessa används finns i slutet av detta dokument
- Exemplen StrBuilder (mutable) och Str (immutable) visar, i förenklad form, hur man skriver en klass för textsträngar
  - Förståelse för hur klasserna StringBuilder och String är implementerade
  - En bra övning i att skriva klasser

Klasser för textsträngar

- Vi vill ha möjlighet att representera textsträngar med funktionalitet …
  - … motsvarande strcat, strcomp, toupper …
- … men till skillnad från i C ska data vara inkapslat, privat …
  - … så att texten endast kan hanteras på ett säkert sätt, till exempel vad gäller minneshantering
  - … heller inget behov av ’\0’

Str(ing) – icke ändringsbara textsträngar

- Objekt av typen Str ska representera en icke ändringsbar, immutable, textsträng
  - Den ursprungliga textsträngen ska, efter den skapats, inte kunna ändras
  - Det ska gå att lägga ihop två textsträngar, concat, men en ny textsträng, den sammanslagna, ska då skapas
  - Det ska, utifrån en existerande textsträng, gå skapa en ny textsträng med versaler, toUpperCase
- Övrig funktionalitet
  - length, charAt
  - Konstruktorer för char, char[]

Str – datamedlemmar och konstruktorer

- Som internt data behövs endast en privat char-array
  - Storlek = antal tecken
  - Lämpligt att deklarera referensen final
- Konstruktorn måste skapa ett eget, internt, array-objekt
  - Om vi enbart kopierar referensen (argumentet) kan anropande kod manipulera arrayen

```
public class Str {

    private final char[] theChars;


    public Str(char ch) {
        theChars = new char[1];
        theChars[0] = ch;
    }


    public Str(char[] chars) {
        theChars = new char[chars.length];

        for (int i = 0; i < theChars.length; i++) {
            theChars[i] = chars[i];
        }
    }
```

Str – length, charAt

- Metoden length returnerar arrayens längd
  - Ingen datamedlem för antal behövs i denna klass
- charAt måste först kontrollera att angivet index inte är felaktigt
  - Kasta exception i annat fall

```
public class Str {

    private final char[] theChars;
    …


    public int length() {
        return theChars.length;
    }


    public char charAt(int index) {
        if (index < 0 || index >= theChars.length)
            throw new IndexOutOfBoundsException(
                "index=" + index);

        return theChars[index];
    }
```

Str – concat, slå ihop textsträngar

- Skapa en ny char-array, tillräckligt stor ...
  - Storleken av this + str (argumentet)
- … och kopiera över alla tecken
  - Två steg
- Anropa konstruktor för att skapa ett nytt Str-objekt
  - Ineffektivt? Konstruktorn skapar en ny array

```
public class Str {

    private final char[] theChars;
    …


    public Str concat(Str str) {
        char[] concatChars =
            new char[this.length()+str.length()];

        for (int i = 0; i < this.length(); i++) {
            concatChars[i] = this.theChars[i];
        }

        for (int j = 0, k = this.length();
             j < str.length(); j++, k++) {
            concatChars[k] = str.theChars[j];
        }

        return new Str(concatChars);
    }
```

Str – toUppercase

- Skapa en ny char-array …
- … och kopiera över alla tecken …
- … men om a-z (a-ö), konvertera a -> A
  - ASCII: A – a = 32
  - å., ä, ö hanteras separat

```
public class Str {

    private final char[] theChars;
    …


    public Str toUppercase() {
        char[] uppercaseChars =
            new char[this.length()];

        for (int i = 0; i < this.length(); i++) {

            char ch = this.theChars[i];
            char newCh;

            if (ch >= 'a' && ch <= 'z')
                newCh = (char) (ch - 32);
            else newCh = ch;

            uppercaseChars[i] = newCh;
        }

        return new Str(uppercaseChars);
    }
```

Str – skapa och använda objekt

```
public static void main(String[] args) {

    Str str = new Str('A');
    System.out.println(str.toString());

    char[] cs = new char[]{'n','d','e','r','s',' '};
    Str str2 = new Str(cs);
    Str str3 = str.concat(str2);
    // str3 -> "Anders" (str, str1 unchanged)

    Str lastName = new Str("Lindström");
    Str str4 = str3.concat(lastName);
    // str4 -> "Anders Lindström" (str2, lastName unchanged)

    Str str5 = str4.toUppercase();
    // str5 -> ”ANDERS LINDSTRÖM" (str3 unchanged)
```

Str – immutable objects

- Objekt av typen Str är immutable eftersom …
  - data är inkapslat och …
  - … vi aldrig lämnar ut refernser till data och …
  - … det inte finns metoder för att ändra i data (mutator-metoder)
- concat, toUppercase returnerar nya Str-objekt
- Det är lämpligt att deklarera datamedlemmen theChars final, men …
  - … final för en referens-variabel säger bara att referensen inte får referera annat objekt/array …
  - … däremot kan elementen i arrayen uppdateras …
  - theChars måste alltså vara privat

```
public class Str {

    private final char[] theChars;

    public int length() { … }

    public char charAt(int index) { … }

    …
    public Str toUppercase() {
        …;
        return new String(…);
    }


    …
}
```

Javas API - klassen java.lang.String

- Naturligtvis använder du klassen java.lang.String för att representera textsträngar
- Eftersom String-objekt används överallt har klassen några speciella egenskaper
  - Immutable => inga problem att dela String-referenser mellan olika objekt
  - Du kan skapa objekt utan att explicit anropa en konstruktor:
    String s = ”Anders”; *// The same as String s = new String(”Anders”);*
  - ”+”-operatorn – slår ihop två textsträngar, concat, och returnerar ett nytt objekt
- Formatera en textsträng med klassmetoden format (C-style)
  - String s = String.format(”The value is %.2f”, value); *// 2 decimals*

En klass för ändringsbara textsträngar, StrBuilder

- Kontrakt för StrBuilder
  - Det ska vara möjligt att lägga till ny text i slutet av en existerande text …
  - … utan att ett nytt objekt skapas
  - Metoder för length, charAt, append, …
- Intern representation
  - En intern char-array, eventuellt med plats för fler tecken än de aktuella …
  - … arrayen behöver utökas om den är fylld

StrBuilder – datamedlemmar och konstruktorer

- Datamedlemmar
  - Privat char-array
  - int n – aktuellt antal tecken …
  - … inte detsamma som arrayens storlek
- Konstruktorn tar ett argument som indikerar önskad kapacitet …
  - … men den interna arrayen har ännu inga tecken
  - Kontrollera indata

```
public class StrBuilder {

    private char[] theChars;
    private int n;

    public StrBuilder(int capacity) {
        if (capacity < 1) {
            throw new IllegalArgumentException(
                "capacity=" + capacity);
        }
        theChars = new char[capacity];
        n = 0;
    }

    public StrBuilder() {
        this(16);
    }
```

StrBuilder – length, charAt

- Metoden length returnerar värdet på n
  - Antalet tecken är inte (inte alltid) detsamma som arrayens längd
- charAt måste först kontrollera att angivet index inte är felaktigt
  - Kasta exception i annat fall

```
public class StrBuilder {

    private char[] theChars;
    private int n;


    …

    public int length() {
        return n;
    }

    public char charAt(int index) {
        if (index < 0 || index >= n) throw new
            IndexOutOfBoundsException(
                "index=" + index);
        return theChars[index];
    }
```

StrBuilder – append

- Metoden append lägger till tecken i slutet av textsträngen
- Kontrollera först att det finns plats i arrayen …
  - … och om så inte är fallet skapa en ny, större array
  - Detta görs i en privat hjälpmetod

```
public class StrBuilder {

    private char[] theChars;
    private int n;
    …

    public void append(char ch) {
        expandIfNeeded(1);
        theChars[n++] = ch;
    }

    …
```

StrBuilder – flera versioner av append

- Metoden append är överlagrad, ”overloaded” …
  - … dvs. det finns tre metoder med samma namn …
  - … men med olika argumentlistor

```
public class StrBuilder {
    private char[] theChars;
    private int n;
    …

    public StrBuilder append(char ch) { … }

    public void append(char[] toAppend) {
        expandIfNeeded(toAppend.length);
        for (int i = 0; i < toAppend.length; i++) {
            theChars[i + n] = toAppend[i];
        }
        n += toAppend.length;
    }
```

StrBuilder – hjälpmetoden expandIfNeeded

1. Kontrollera om arrayen behöver utökas
2. Skapa i så fall en ny array, med plats för de nya elementen och mer (här: dubbla)
3. Kopiera över elementen (tecknen) till den nya arrayen
4. Låt datamedlemmen theChars referera den nya arrayen
5. Den tidigare arrayen kommer att bli ”garbage collected”

```
public class StrBuilder {
    private char[] theChars;
    private int n;

    public StrBuilder append(char[] toAppend) {
        expandIfNeeded(toAppend.length);
        …
    }

    private void expandIfNeeded(int addedChars) {
        if (n + addedChars >= theChars.length) {
            char[] tempArr =
               new char[2 * (n + addedChars)];
            for (int i = 0; i < n; i++) {
                tempArr[i] = theChars[i];
            }
            theChars = tempArr;
        }
    }
```

StrBuilder – skapa och använda objekt

```
public static void main(String[] args) {

    StrBuilder builder = new StrBuilder();

    builder.append('A');
    System.out.println(builder.toString());

    char[] cs = new char[]{'n','d','e','r','s',' '};
    builder.append(cs);
    System.out.println(builder.toString());

    builder.append("Lindström");
    System.out.println(builder.toString());
}
```

Javas API - klassen java.lang.StringBuilder

- Om du behöver ”slå ihop” många del-texter när du genererar en längre textsträng är klassen String ineffektiv
  - Varje ”+”, ”+=” eller concat genererar ett nytt String-objekt …
  - … och tidigare objekt blir ”garbage collected”
- Använd då istället en StringBuilder i stället - mutable
  - append
  - insert
  - replace
  - …

Javas API - klassen java.lang.StringBuilder

- Skapa en textsträng med av alla elementen i en array
  - [element1, element2 , ….]
- Ett enda String-objekt skapas på slutet
  - Om vi istället använt String och +/concat hade en array med 100 element genererat 100 String-objekt …
  - … varav endast det sista använts

```
StringBuilder builder = new StringBuilder();

builder.append("[");

if (elements.size() > 0) {
    builder.append(elements[0]);
    for (int i = 1; i < n; i++)
        builder.append(", ").append(elements[i]);
}

builder.append("]");

String result = builder.toString();
```